AI Engineering and Claude Architect · Part V, Certification

Certification, Mock Exam 3

60 Items in 120 Minutes

Scenarios: Code Generation with Claude Code, Multi-Agent Research System, Developer Productivity with Claude, Claude Code for Continuous Integration. Set a timer, work without notes, and mark your answers on paper.

Items	60, in 4 scenario sets of 15
Time	120 minutes
Domains	16, 11, 12, 12 and 9 items, rounded from the guide's weights
Status	Original practice items, written from the guide. Not real or leaked

Before You Start

Key idea

These are original practice items, not real or leaked exam questions, and the company names are invented. A score here doesn't predict your exam result.

Follow the real conditions: one sitting, no notes, flag and return. Where the guide and current products differ, the answers follow the guide, and an explanation may add a Build today note. The real exam's wording may be longer and harder, as candidates report.

Checkpoints: 28 minutes after Scenario 1, 56 after Scenario 2, 84 after Scenario 3 and 110 after Scenario 4.

Targets, which are my own and not official: 80% overall, and no domain under 70%.

Scenario: Code Generation with Claude Code

Key idea

Larkspur Software's 25 developers use Claude Code across a Spring Boot backend and a TypeScript frontend. They want consistent code generation, reliable refactoring and useful documentation, and they often run long sessions on an old codebase.

Q1. Which location holds instructions that only one developer receives, and that teammates never get through version control?

A. The repository's root `CLAUDE.md`
B. `.claude/CLAUDE.md` in the repository
C. A `CLAUDE.md` in a subdirectory
D. `~/.claude/CLAUDE.md`

Q2. The backend and frontend packages have different standards, and the shared standards live in `docs/standards/`. What's the guide's modular way to include only what each one needs?

A. A single user-level file containing the standards for both of the packages
B. A copy of all the standards files pasted into each package's `CLAUDE.md`
C. `@import` of the relevant standards files, in each package's `CLAUDE.md`
D. A slash command that developers must remember to run in each new session

Q3. Which belongs in a skill, rather than in `CLAUDE.md`?

A. The rule that all public methods need documentation
B. The team's naming convention used in all generated code
C. A step-by-step release-notes workflow used about once a month
D. The project's build and test commands, used every day

Q4. A brainstorming skill produces many rejected ideas, which then bias later work in the session. What's the best frontmatter change?

A. `context: fork`, so the exploration stays out of the main conversation
B. `paths`, so the skill runs only on files that have brainstorming content
C. `argument-hint`, so developers provide a narrower topic for brainstorming
D. `allowed-tools: Write`, so the skill records its ideas to a file only

Q5. Why might a `.claude/rules/` file with `paths: ["**/*.test.ts"]` be better than a large `CLAUDE.md` section on tests?

A. It's stored in each user's home directory, so developers can each change it
B. It runs as a hook, so each test is executed automatically after any edit
C. It loads only when matching files are edited, which reduces irrelevant context
D. It's applied by the API, so the model can never fail to follow what it says

Q6. Which is the best fit for plan mode?

A. Choosing how to restructure a monolith module into services, with several valid approaches
B. Renaming one local variable within a single short function
C. Fixing a typo in a log message whose location is known
D. Adding a date check to a function that's already been identified by a failing test

Q7. A multi-phase refactor keeps running out of context during discovery, before implementation begins. What's the best approach?

A. Put the full source for each file into `CLAUDE.md` for the rest of the phases
B. Use the Explore subagent for discovery, keeping its summaries in the main session
C. Ask Claude to skip discovery, and work from what's typical for Spring Boot projects
D. Turn off plan mode, so that discovery goes straight into making the changes

Q8. A developer describes a CSV transformation in prose, and Claude's output differs on each attempt. What should they do next?

A. Switch to a much shorter prose description
B. Ask Claude to explain the prose back to them
C. Repeat the same prose with stronger emphasis
D. Provide 2–3 concrete input and output examples

Q9. Three independent bugs exist in unrelated modules. How should the developer give feedback, and why?

A. All at once, because a single message always gives the best results
B. All at once, because Claude can only remember one message at a time
C. Sequentially, because fixes for independent issues don't interact
D. Sequentially, because a session can contain just one fix for each file

Q10. An API-based helper "forgets" the previous question on each turn. What's the most likely cause?

A. The system prompt is limiting the amount that the model can recall
B. The model's memory fills up after each response has been returned
C. `stop_reason` was `end_turn`, which erases the server's stored memory
D. The client isn't sending the complete conversation history in each request

Q11. A long session's progressive summary changes "retry 3 times, then fail after 30s" into "retry a few times". Which principle does this show, and what prevents it?

A. Summaries lose only customer details, so this technical detail is safe to lose in the session
B. Summaries lose specifics, so keep such values in a persistent facts block outside the summary
C. Long sessions always use a smaller model, so select a larger model at the start of the session
D. Models round all numbers by design, so an instruction not to round is what prevents it

Q12. A developer asks about a class found an hour ago, and Claude answers with generic advice. What should the agent have been doing during exploration?

A. Avoiding reading any files, so that its context stays as small as possible
B. Using a smaller `max_tokens`, so that answers don't exceed a limit
C. Re-reading every file in the repository before answering each question
D. Maintaining a scratchpad file of key findings, and referring back to it

Q13. Between two phases of a large investigation, what does the guide recommend?

A. Keep all raw output from the first phase in the main session for later phases
B. Summarize key findings from one phase, and inject them into the next phase's subagents
C. Discard all findings, so that each phase can look at the code without any bias
D. Run the phases in reverse order, so that the later findings are considered first

Q14. An overnight multi-agent migration analysis crashes. What should recovery rely on, by the guide?

A. A fresh run of the entire analysis, which is always more reliable than any recovery
B. Each agent's exported state in a known location, plus a manifest the coordinator loads on resume
C. The model's memory of its earlier work, which resumes automatically once the process has been restarted
D. The final report file, which holds the complete working state of every agent involved

Q15. What's the main value of delegating "find all usages of this config key" to a subagent?

A. The subagent can use tools without any restrictions at all
B. The subagent has access to memory from all earlier sessions
C. The main agent's model automatically becomes faster for later work
D. The verbose output stays out of the main agent's context

Scenario: Multi-Agent Research System

Key idea

Quillstone Research runs a multi-agent system on the Agent SDK. The coordinator delegates to specialists for web search, document analysis, synthesis and report writing. Clients rely on its cited reports for investment decisions.

Q16. Which value should keep the coordinator's loop running?

A. The response includes some text
B. `stop_reason` is `tool_use`
C. The response is shorter than usual
D. `stop_reason` is `end_turn`

Q17. What's isolated about a subagent's context in this system?

A. It shares context with its sibling subagents, but not with its coordinator
B. It can't receive anything at all from the coordinator, including findings
C. It doesn't automatically inherit the coordinator's conversation history
D. It must run on a different model from the one the coordinator is using

Q18. Choose the coordinator responsibilities described in the guide. **Select 2.**

A. Task decomposition and delegation
B. Writing each subagent's tool code at runtime
C. Holding the shared memory that all subagents read directly
D. Aggregating results, and deciding which subagents to invoke
E. Letting specialists resolve errors among themselves

Q19. Which `AgentDefinition` fields does the guide highlight?

A. API key, region and billing account
B. Description, system prompt and tool restrictions
C. Temperature, seed, stop sequences and retries
D. Database URL, cache size and timeout

Q20. Which prompt best prepares the synthesis subagent?

A. A list of numbered steps for synthesis, with a request to look up what earlier agents found
B. The goal and quality criteria, plus complete findings as structured data with source metadata
C. "Synthesize the findings from earlier", relying on the coordinator's history being there
D. The goal only, as a subagent automatically knows what its sibling subagents have discovered

Q21. A new client topic is broad and poorly defined, and the useful directions aren't known up front. Which decomposition fits?

A. None: send one prompt with the whole topic to a single subagent
B. Fixed: use the same five subtopics as last month's unrelated report
C. Dynamic: let intermediate findings shape subsequent subtasks
D. Reverse: write the report, then look for sources that support it

Q22. An analyst wants to explore a bull case and a bear case from one shared analysis. What's the right session approach?

A. Resume the same session for both cases
B. Use `-p`, and rely on the output being shared
C. Fork from the analysis, once per case
D. Use `/compact`, and write both in one session

Q23. A compliance policy says no report may be sent until a source check has passed. Which implementation is appropriate?

A. The report subagent includes a line stating that sources have been checked
B. A programmatic gate blocks the send step until the check's result is recorded
C. The coordinator's system prompt asks it to always run the check before sending
D. A few-shot example shows the check being run before a report is sent

Q24. A generic `analyze_document` tool gives uneven results. Which change follows the guide?

A. Add a required `instructions` string, so that callers can say exactly what they'd like
B. Split it into `extract_data_points`, `summarize_content` and `verify_claim_against_source`
C. Rename it `analyze_document_v2`, and keep its description exactly as it was before
D. Add three more generic tools, so that Claude has more choices for each document

Q25. Which responsibilities belong to a search subagent facing a transient failure?

A. Report success, and leave out the failure, so that the coordinator's plan isn't disrupted
B. Stop the whole workflow straight away, so that no partial data can enter the report
C. Recover locally, and propagate only what it can't resolve, with partial results and attempts
D. Propagate every failure at once, so that the coordinator can make every retry decision

Q26. Why is limiting agents to 4-5 tools, rather than 18, an example of a principle rather than a rule?

A. The point is that scope affects selection reliability, and the number is only an illustration
B. The exam guide has a table setting a maximum number for each of the agent types
C. The API rejects a request whenever it contains a tool list longer than exactly five entries
D. MCP servers can only ever expose four or five tools at any one time

Q27. Tools from three configured MCP servers appear for the agent. When are they discovered, and how are they available?

A. At deployment, and only one server is active per session
B. On each request, one server at a time, in alphabetical order
C. Only after a model has requested a server by name
D. At connection time, and all are available simultaneously

Q28. Which subagent message gives the coordinator the most useful information?

A. `{"status":"search unavailable","message":"The search service could not be reached. Please try again."}`
B. `{"error":true}`
C. `{"status":"ok","results":[]}`
D. `{"failure":"timeout","query":"EU tariffs 2026","partial":[2 results],"alternatives":["trade database"]}`

Q29. A report needs earnings figures, market news and a technical analysis. How should the synthesis render these?

A. All of it as bullet points, so that the whole report is uniform throughout
B. Financial data as tables, news as prose, and technical findings as structured lists
C. All of it as tables, so that readers can compare any item with any other
D. All of it as prose, so that the report reads smoothly from start to finish

Q30. A source says "preliminary estimate", and the synthesis writes the figure as established fact. What should change?

A. Ask the report subagent to add "approximately" to every figure in every report that it produces from now on
B. Preserve source characterizations and separate well-established findings from contested ones
C. Exclude all preliminary sources, including the only sources on a given topic
D. Remove all qualifiers from reports, so that clients can read them more easily

Scenario: Developer Productivity with Claude

Key idea

Pinecrest Bank's platform team builds a developer productivity agent on the Agent SDK. It explores legacy Java services, generates boilerplate and automates repetitive changes. It uses built-in tools and connects to Jira and an internal documentation service over MCP.

Q31. What makes this agent model-driven, as opposed to a pre-configured tool sequence?

A. The agent runs on the newest available Claude model
B. The tool order is held in a configuration file that the team can edit
C. Claude decides which tool to call next, based on what it has already learned
D. Every step is approved by a human before the tool runs

Q32. A coordinator always runs exploration, generation, testing and documentation, even for "what does this class do?" What's the guide's lesson?

A. Analyze what the query needs, and invoke only the relevant subagents
B. Have the user choose the subagents, as coordinators can't analyze a query
C. Always run all subagents, so that results are consistent across all queries
D. Add a caching layer, so that the full pipeline is cheaper on simple queries

Q33. Which statement is correct about spawning subagents in the guide's terms?

A. Only one subagent can run in any session
B. Subagents share the coordinator's memory automatically
C. Each subagent has to be a separate MCP server
D. The coordinator's `allowedTools` must include `"Task"`

Q34. Automation must never run `Bash` commands that touch the production database, and failures should be redirected to a ticket. Which is most appropriate?

A. A PostToolUse hook that reports the command to the platform team, once it has finished running
B. A tool-interception hook that blocks such commands, and redirects to a ticket workflow
C. A subagent whose only job is reading the logs for any production access
D. A note in `CLAUDE.md` asking the agent never to touch production data

Q35. Which task is the best fit for a PostToolUse hook?

A. Normalizing mixed timestamp formats from two MCP tools
B. Preventing a tool from running above a certain limit
C. Choosing which subagent should handle a user's query
D. Writing the final answer for the engineer to read

Q36. A review of a large generated change misses cross-module bugs, and says conflicting things about similar code. Which decomposition does the guide support?

A. Analyze each file individually, then run a cross-file integration pass
B. Analyze only the largest file, and assume the other files are similar
C. Analyze the files in random order, then keep only the first findings
D. Analyze all the files at once, and ask the model to take more care

Q37. When should an engineer prefer resuming over starting fresh?

A. When most prior tool results are stale
B. When the code has been heavily rewritten
C. When they want an independent reviewer
D. When prior context is mostly still valid

Q38. Exploration subagents return overlapping dependency analyses. Which is the coordinator's best fix?

A. Remove all but one subagent, however large the codebase turns out to be
B. Tell the subagents to compare their results directly with each other
C. Assign each subagent a distinct area, such as a service or dependency type
D. Discard any finding that appears in more than one subagent's output

Q39. Which tool description best follows the guide?

A. "Searches Jira issues by JQL. Input: a JQL string. Returns: key, summary, status. Use for issues, not documentation."
B. "Jira search."
C. "Searches for things in Jira. This tool is very powerful and is useful for a wide variety of tasks you may have."
D. "Use this tool whenever you need information of any sort, because it's the best tool available to the agent."

Q40. Jira returns a permission error for a restricted project. What should the MCP tool do?

A. Return `isError` with a permission category, marked not retryable, and describe what's wrong
B. Retry until it succeeds, since permissions are often restored in a moment
C. Return text saying "Operation failed", because further details aren't useful to the agent in this case
D. Return an empty list, which keeps the agent from raising unnecessary alarms

Q41. The team needs a Jira integration. What does the guide suggest first?

A. Use an existing community MCP server, and reserve custom servers for team-specific workflows
B. Give the agent `Bash`, and have it write a new Jira client for each request
C. Build a custom MCP server, because custom servers always fit a team's needs better than community ones
D. Skip MCP, and paste the Jira data into `CLAUDE.md` each morning before work

Q42. Which belongs in `.mcp.json`?

A. `"Authorization": "see the wiki page for the token"`

B. `"Authorization": "Bearer abc123-real-token"`

C. `"Authorization": "Bearer ${JIRA_TOKEN}"`

D. `"Authorization": "read ~/.claude/CLAUDE.md"`

Q43. Which sequence best builds understanding of a legacy flow?

A. Grep for the entry point, then Read and follow imports to trace the flow

B. Glob for all Java files, then Write a summary of each, then read those

C. Read every file in the repository, then Grep for whatever seems relevant

D. Edit the entry point to add logging, then run it in the production system

Q44. An agent needs to find where an error message is produced, and then see how the function is called. Which tools fit? **Select 2.**

A. Grep for the function's name to find its callers

B. Write to create an index of error messages

C. Edit to search through file contents

D. Glob for the error message text

E. Grep for the error message text

Q45. You want to be sure the agent responds with a tool call and not conversational text, but it may choose the tool. Which setting fits, in the guide's terms?

A. `max_tokens: 1`

B. `tool_choice: {"type": "tool", "name": "search"}`

C. `tool_choice: "any"`

D. `tool_choice: "auto"`

Scenario: Claude Code for Continuous Integration

Key idea

Ironwood Commerce runs Claude Code in its pipeline. It reviews every pull request before merge, runs a nightly security audit, generates tests and posts findings to developers. Findings must be precise and actionable.

Q46. Which is the mechanism for giving CI-invoked Claude Code its testing standards and review criteria?

A. A CI runner's `~/.claude/CLAUDE.md`
B. A long `--resume` session history
C. `CLAUDE.md` in the repository
D. An environment variable holding each rule

Q47. Which pipeline command is best for automated, parseable, non-interactive review?

A. `claude "..." --output-format text`
B. `claude -p "..." --output-format text --append-system-prompt "Reply in JSON"`
C. `claude -p "..." --output-format json --json-schema "$SCHEMA"`
D. `claude --resume last`

Q48. Reviewers want the same review procedure available to everyone, through the repo. Which location is right, in the guide's terms?

A. `~/.claude/skills/review/SKILL.md` or `~/.claude/commands/review.md`
B. `~/.claude/commands/review.md`
C. `.claude/skills/review/SKILL.md` or `.claude/commands/review.md`
D. `~/.claude.json`

Q49. Which instruction is an explicit criterion?

A. "Report issues a senior engineer would care about."
B. "Be conservative about what you report here."
C. "Report only the issues that you're highly confident about, and leave out everything else."
D. "Report SQL built by string concatenation with user input. Skip formatting issues."

Q50. Severity labels vary between runs for the same issue. What's the best fix?

A. Define each level, with concrete code examples
B. Remove severity labels from all findings entirely
C. Average severity across three different runs
D. Add a sentence asking the model to be consistent

Q51. What's the guide's main concern about a high false-positive category?

A. It costs much more than other categories to run through the API
B. It breaks the JSON schema used for the review's findings
C. It makes the batch API stop processing the job in progress
D. It undermines developers' confidence in accurate categories as well

Q52. Which few-shot example design most reduces false positives?

A. One example of a clean file, and nothing else
B. Pairs showing an acceptable pattern and a similar genuine issue, with the reasoning
C. Several examples of the same bug in different languages
D. A dozen obvious bugs, along with the exact output format that's expected for each of them

Q53. Findings are correct, but in varying formats that are hard to act on. What's the guide's most effective technique?

A. Few-shot examples showing location, issue, severity and suggested fix
B. A larger model, with no changes to the prompt at all
C. Longer prose instructions that describe the desired output for each finding
D. A request to write the findings out with greater care

Q54. The schema marks `suggested_fix` as required, but some findings have no clear fix, and invented fixes have appeared. What should change?

A. Add a minimum string length for that field
B. Make `suggested_fix` nullable
C. Remove findings that have no clear fix
D. Add "be accurate" to the system prompt

Q55. Which schema field supports later analysis of why developers dismiss findings?

A. `stop_reason`
B. `isRetryable`
C. `custom_id`
D. `detected_pattern`

Q56. Which job suits the Message Batches API?

A. An interactive debugging session
B. The check that blocks each merge
C. A developer's live question
D. The nightly security audit

Q57. A manager wants to save 50% by moving pre-merge reviews to batch. What's the best response?

A. Batch has no latency guarantee and can take up to 24 hours, so blocking checks should stay synchronous
B. Batch is fine for pre-merge reviews, as long as developers submit a smaller number of files in each pull request
C. Batch is fine for pre-merge reviews, as it's always finished within a few minutes
D. Batch can't be used for any code analysis, as it doesn't support the code in requests

Q58. Why can't a batch request carry out a multi-turn audit that runs tools as it goes?

A. Batches don't accept any `tools` parameter in a request, in any form

B. Batches require a human to approve each of the tool calls that are made during the audit

C. Batches are limited to one document of a very small size for each job

D. Batches can't execute tools mid-request and return the results within that request

Q59. Why is a second, independent Claude instance better at review than asking the generator to double-check itself?

A. It doesn't carry the generator's reasoning context, so it's less inclined to accept prior decisions

B. It runs with extended thinking, which self-review is not able to use at all

C. It always runs on a stronger model, whichever configuration has been selected

D. It has access to the generator's private thinking, so it can see where the generator's mistakes came from

Q60. A team wants to skip per-file analysis for large PRs, and just use extended thinking on one pass. What's the guide's position?

A. A single pass is enough if its output is restricted by a schema that requires per-file sections

B. Large PRs shouldn't be reviewed by a model, so the team should send them only to human reviewers

C. Extended thinking fully replaces multi-pass review, so it's the simplest and most effective answer for large changes

D. Split them into per-file passes plus integration passes, to avoid attention dilution and contradictory findings

Answer Key and Score Sheet

Q	Ans	Q	Ans	Q	Ans	Q	Ans
1	D	2	C	3	C	4	A
5	C	6	A	7	B	8	D
9	C	10	D	11	B	12	D
13	B	14	B	15	D	16	B
17	C	18	A, D	19	B	20	B
21	C	22	C	23	B	24	B
25	C	26	A	27	D	28	D
29	B	30	B	31	C	32	A
33	D	34	B	35	A	36	A
37	D	38	C	39	A	40	A
41	A	42	C	43	A	44	A, E
45	C	46	C	47	C	48	C
49	D	50	A	51	D	52	B
53	A	54	B	55	D	56	D
57	A	58	D	59	A	60	D

Domain	Items	Your score	Target
1. Agentic Architecture and Orchestration	16		70%+
2. Tool Design and MCP Integration	11		70%+
3. Claude Code Configuration and Workflows	12		70%+
4. Prompt Engineering and Structured Output	12		70%+
5. Context Management and Reliability	9		70%+
Total	**60**		**80%+**

The guide doesn't publish how raw scores map to the scaled 720, so these targets are only study guidance.

Your error log: item, what you picked, why it was wrong

Explanations

1. **D** (TS 3.1). User-level instructions apply only to that user. The others are in the repository, so they're shared.

2. **C** (TS 3.1). `@import` keeps files modular and selective. Build today: the current syntax is written `@path`. Copies drift, commands get forgotten, and user-level isn't shared.

3. **C** (TS 3.2). Skills suit on-demand, task-specific workflows. Always-relevant standards belong in `CLAUDE.md`.

4. **A** (TS 3.2). Isolating exploratory context is what `context: fork` is for. The others don't stop the output from entering the session.

5. **C** (TS 3.3). Path-scoped rules are conditional context. They're still instructions, not enforcement, and they aren't hooks.

6. **A** (TS 3.4). Architecture and multiple approaches call for plan mode. The rest are simple, well-scoped changes.

7. **B** (TS 3.4). Explore isolates verbose discovery and returns summaries. Skipping discovery invites errors, and pasting source wastes context.

8. **D** (TS 3.5). Examples are the guide's answer when descriptions are interpreted inconsistently.

9. **C** (TS 3.5). Independent issues can go one at a time. Together is for interacting problems. The other reasons are false.

10. **D** (TS 5.1). The Messages API needs the history each time. Coherence depends on passing it. `end_turn` erases nothing.

11. **B** (TS 5.1). Progressive summarization risks condensing numbers. A separate facts block preserves them. The other claims aren't true.

12. D (TS 5.4). A scratchpad persists findings across context boundaries. Rereading everything reintroduces the problem.

13. B (TS 5.4). Summarizing, then injecting, preserves findings without the raw output. The other approaches waste context or lose work.

14. B (TS 5.4). Structured exports and a manifest let the coordinator resume and inject state into prompts. The others either lose work, or assume state that isn't there.

15. D (TS 5.4). Isolation is the value. Subagents don't have those other properties by default.

16. B (TS 1.1). `tool_use` means tools must run and the loop continues. `end_turn` ends it. Text and length aren't signals.

17. C (TS 1.2). Isolation means no automatic inheritance. Findings can still be passed in the prompt.

18. A and D (TS 1.2). The guide names decomposition, delegation, aggregation and selection, with all communication and error handling going through the coordinator.

19. B (TS 1.3). The guide names descriptions, system prompts and tool restrictions. The others aren't what it highlights.

20. B (TS 1.3). It needs goals, criteria and the complete findings. Nothing is inherited, and procedural steps reduce adaptability.

21. C (TS 1.6). Open-ended work suits adaptive decomposition. A fixed list ignores discovery, and writing first would bias the sources.

22. C (TS 1.7). Forks give independent branches from a shared baseline.

23. B (TS 1.4). A required order needs a programmatic prerequisite. The others are probabilistic, and a statement in a report proves nothing.

24. B (TS 2.1). Purpose-specific tools with defined contracts are the guide's approach. A free-form parameter keeps the tool generic.

25. C (TS 2.2). Local recovery with structured propagation is the guide's approach. The others are its anti-patterns, or overload the coordinator.

26. A (TS 2.3). The guide's example illustrates decision complexity. It isn't a protocol limit.

27. D (TS 2.4). The guide says tools from all configured servers are discovered at connection and available simultaneously.

28. D (TS 5.3). Failure type, query, partial results and alternatives enable recovery. Generic statuses hide context, and empty success is misleading.

29. B (TS 5.6). The guide says to render content types appropriately, instead of converting everything to a uniform format.

30. B (TS 5.6). The guide requires preserving characterizations and methodological context, with explicit sections for established and contested findings.

31. C (TS 1.1). The guide defines this distinction by who decides the next action.

32. A (TS 1.2). Dynamic selection avoids unnecessary subagents. Caching keeps the wasted pipeline.

33. D (TS 1.3). The guide states the first, and it says subagents don't inherit context. Build today: the tool is named `Agent`.

34. B (TS 1.5). A pre-execution block is a guarantee. PostToolUse runs too late, a note is guidance, and log reading is detection only.

35. A (TS 1.5). PostToolUse transforms results before the model processes them. Blocking must happen before execution.

36. A (TS 1.6). Per-file plus integration avoids attention dilution. The others ignore the problem.

37. D (TS 1.7). Resume when context is mostly valid. Stale results call for a fresh session with a summary, and independent review needs a separate instance.

38. C (TS 1.2). Partitioning minimizes duplication. Direct comparison breaks routing, and agreement isn't a reason to discard.

39. A (TS 2.1). It states purpose, inputs, outputs and boundaries. The others are vague, minimal or overlapping.

40. A (TS 2.2). Permission errors are non-retryable, and they need a clear description. An empty list falsely implies no issues.

41. A (TS 2.4). Standard integrations should use community servers. Custom servers are for team-specific needs.

42. C (TS 2.4). Environment expansion keeps credentials out of version control. A real token would be committed, and the others don't work.

43. A (TS 2.5). The guide describes incremental understanding, rather than reading everything upfront.

44. A and E (TS 2.5). Grep searches contents. Glob matches paths, and Write and Edit modify files.

45. C (TS 2.3). `any` requires some tool while leaving the choice open. Build today: it's rejected on some current models.

46. C (TS 3.6). The guide names `CLAUDE.md` as the mechanism. Runner home files aren't shared in the repo. Build today: `--bare` skips `CLAUDE.md`.

47. C (TS 3.6). It's non-interactive, structured and schema-checked. The others wait for input or return text.

48. C (TS 3.2). Project-scoped files are shared. The others are personal.

49. D (TS 4.1). It defines what to report and what to skip. The rest are vague, or based on confidence.

50. A (TS 4.1). Concrete criteria and examples give consistent classification.

51. D (TS 4.1). False positives erode trust across categories. The other claims aren't what's described.

52. B (TS 4.2). Distinguishing acceptable from genuine, with reasoning, supports generalization.

53. A (TS 4.2). Few-shot examples are the guide's answer when instructions alone give inconsistent format.

54. B (TS 4.3). Nullable removes the pressure to fabricate. A minimum length makes it worse, and dropping findings loses valid issues.

55. D (TS 4.4). `detected_pattern` records which construct triggered a finding. The others serve other purposes.

56. D (TS 4.5). Batches fit non-blocking, latency-tolerant work, as there's no latency guarantee.

57. A (TS 4.5). Blocking workflows don't suit batch. The other statements misstate what batch does.

58. D (TS 4.5). The guide says batches don't support multi-turn tool calling within a request.

59. A (TS 4.6). Independence is the point. The guide says independent instances beat self-review instructions and extended thinking.

60. D (TS 4.6). Multi-pass is the guide's design. A schema shapes the output without fixing the attention problem.